ARM AI OPTIMIZATION CHALLENGE | CLOUD AI

PerfRunbench

Guarded autotuning for repeatable AI workloads

Bounded search, statistical confirmation, and safe recommendation reuse.



REAL ARM64 DATA | GENERIC HARDWARE VISUAL

3 sizes
C7g measured

1,047
HTTP measurements

514 / 0
artifact checks / failures

C7g.xlarge to C7g.4xlarge | Qwen2.5-7B Q4_K_M | pinned llama.cpp runtime | guarded concurrency 4

01 | THE ACTUAL PROBLEM

The fastest run is not automatically the safest recommendation.

The best setting depends on the target, workload, runtime, and measurement protocol.

The fastest point may be noise.

The environment may drift.

One workload may improve while another regresses.

RUNTIME SEARCH — 5% noise guard

`candidate`

observed

`paired confirmation`

eligible

Fastest observed point is not a global optimum.

WORKLOAD GUARD — same profile

interactive + decode	eligible
different traffic shape	
prefill-heavy	rejected

ENVIRONMENT VALIDATION — hard drift

`C7g recommendation on T4g: refused`

`failed runs remain failures`

No recommendation is manufactured.

02 | PRODUCT INTERACTION

One command starts a bounded, process-scoped tuning run.

CONFIG-FREE ENTRY

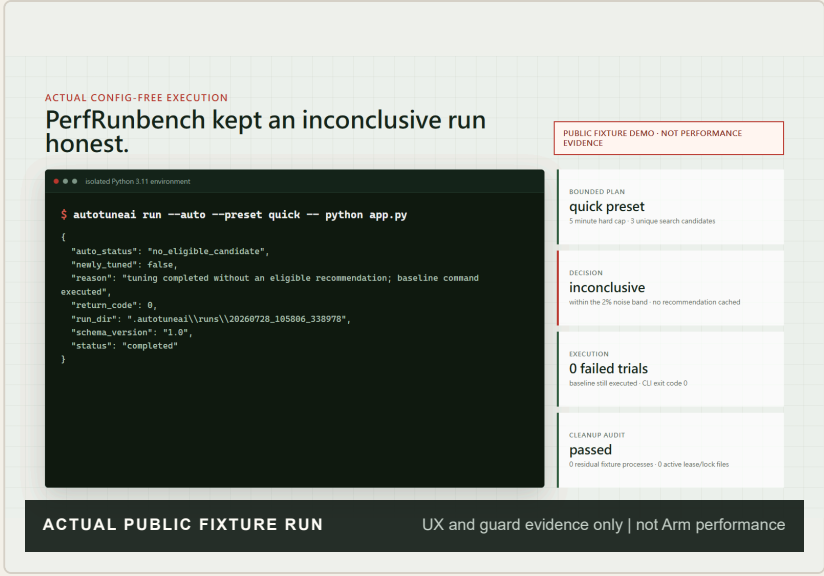
```
$ autotuneai run --auto --preset quick -- python app.py
```

quick 5 min / 3 candidates **standard** 20 min / 8 candidates **deep** 60 min / 16 candidates

01 **No configuration file**

02 **Hard execution budget**

03 **No machine-wide state change**



When the hard budget expires, PerfRunbench still writes a valid partial report and runs cleanup. Invalid evidence is not cached.

03 | GUARDED PIPELINE

Every recommendation carries the evidence needed to reuse it safely.

01 **Fingerprint**
Command, target, runtime

02 **Search**
Deterministic plan, hard budget

03 **Repeat**
Warm-up exclusion, paired runs

04 **Confirm**
Noise band, failures, samples

05 **Passport**
Digests, provenance, statistics

06 **Guarded reuse**
Validate, canary, lease, rollback

Measure once. Reuse only when identity still matches.

- × parser failure
- × inside noise band
- × hard drift
- × cleanup failure

04 | SAME-PROTOCOL HARDWARE SCALING

One pinned workload scales from 3 to 15 Arm server threads.



C7G.XLARGE | 3 THREADS

16.444 / 7.101

prefill / decode tok/s

C7G.2XLARGE | 7 THREADS

38.993 / 15.541

prefill / decode tok/s

C7G.4XLARGE | 15 THREADS

73.598 / 29.871

prefill / decode tok/s

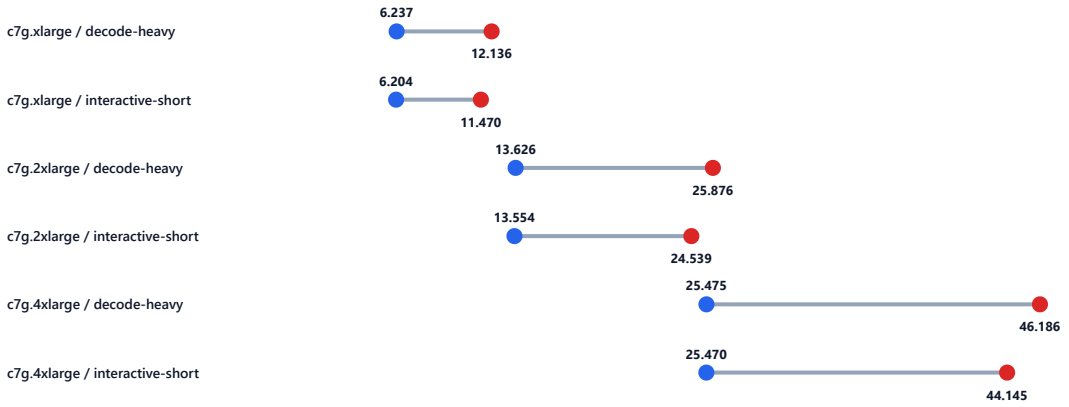
pp512 and tg128 are separate llama-bench tests | one warm-up plus five measured repetitions per point | one client CPU reserved | absolute scaling is not added to runtime-tuning percentages.

05 | BEFORE / AFTER ACROSS THREE SIZES

Four-slot continuous batching improved interactive and decode serving.

Serving before and after by C7g size

Formal paired confirmation, concurrency 4. Blue is one-slot baseline; red is four-slot continuous batching.



XLARGE DECODE

6.237 to 12.136

+94.6%

2XLARGE DECODE

13.626 to 25.876

+89.9%

4XLARGE DECODE

25.475 to 46.186

+81.3%

Formal paired confirmation | three randomized pairs | fresh server restart between blocks | 20 measured requests per workload and side | 10,000-sample bootstrap intervals exclude zero.

06 | WORKLOAD-AWARE GUARD

The tuner refuses a profile when the traffic shape changes.

Workload-aware screening guard

Green cells cleared the 5% screening threshold. Red cells were rejected; prefill-heavy also triggered TTFT regression.

	c7g.xlarge		c7g.2xlarge		c7g.4xlarge	
	slots2	slots4	slots2	slots4	slots2	slots4
interactive-short	-0.6% rejected	+73.7% eligible	+0.2% rejected	+67.4% eligible	+7.0% eligible	+67.8% eligible
decode-heavy	+0.4% rejected	+84.5% eligible	+1.3% rejected	+77.9% eligible	+5.7% eligible	+72.0% eligible
prefill-heavy	-23.0% rejected	-39.0% rejected	-22.3% rejected	-38.0% rejected	-18.8% rejected	-35.3% rejected

6 / 6 formal interactive + decode comparisons eligible

3 / 3 four-slot prefill-heavy slices rejected

WHY REJECTED

-35.3% to -39.0%

Throughput regressed and p95 TTFT worsened. No global recommendation was created.

The selected family is eligible only for interactive-short and decode-heavy under this model, target size, concurrency, and protocol.

07 | REAL REMOTE SERVING

69.4% to 86.5% more remote output throughput.



Validated remote topology 4 parallel slots | continuous batching | C7g server | T4g client

direct private path | concurrency 4 | 608 measured requests | 0 failures

08 | STATISTICAL GUARD

A trustworthy tuner must be able to say no.

Measured is not eligible

PerfRunbench recommends only candidates that clear evidence, provenance, correctness, and cleanup gates.

NO ELIGIBLE RECOMMENDATION

Prefill-heavy serving	regressed across all three C7g sizes
Later compiler / backend rerun	inside 5% noise band
no-mmap	+3.014% | below guard
Network tuning	not executed | no bottleneck evidence

Negative evidence is preserved. Unsupported, regressed, or inconclusive candidates remain visible instead of becoming a speedup claim.

09 | EVIDENCE, NOT JUST NUMBERS

Performance results remain bound to identity, protocol, and cleanup.

MEASURED ON ARM64

3 C7g sizes

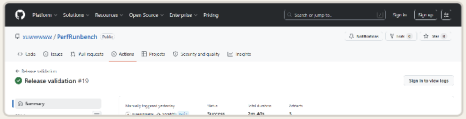
Arm Neoverse-V1 / Graviton3

- same model and binary digests
- 1,047 measured HTTP requests
- 30 fresh server starts

INDEPENDENT VERIFICATION

Artifacts	514 checked	**PASS**
HTTP	0 failures	**PASS**
Cleanup	all targets	**PASS**

Performance Passport status is eligible preliminary. The result is not a global optimum.



Compatibility stays separate from performance

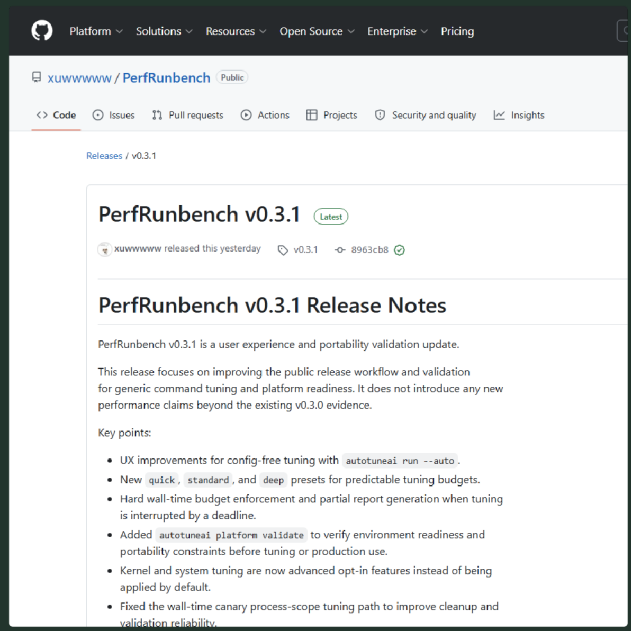
Windows x86-64, Ubuntu x86-64, and Ubuntu Arm64 installed-wheel validation

PERFRUNBENCH | v0.3.1

Measure once. Reject false gains. Reuse safely.

Best observed eligible configuration under the tested target, model, workload, runtime, concurrency, topology, and measurement protocol.

REPOSITORY	github.com/xuwwwww/PerfRunbench
RELEASE	github.com/xuwwwww/PerfRunbench/releases/tag/v0.3.1



Published GitHub Release | verified wheel and sdist digests

@xuwwwww | Arm AI Optimization Challenge | Cloud AI

OPEN SOURCE | MIT LICENSE